

Reinforcement Learning Course: Week 2 Summary

The Deep RL Era: Value-Based Approximators and Deep Q-Networks (DQN)

1 Introduction

During Week 1, we explored tabular methods where state-action values were stored inside fixed matrices (Q-Tables). However, tabular methods suffer from a severe limitation known as the curse of dimensionality: when environments become large, continuous, or handle raw sensory inputs (like pixels or complex physical coordinates), maintaining a table is practically impossible due to memory explosion.

Week 2 transitions into Deep Reinforcement Learning (DRL). By utilizing Deep Neural Networks as non-linear function approximators, agents can generalize past experiences to unseen states, forming the foundation of modern value-based algorithms such as Deep Q-Networks (DQN).

2 From Q-Tables to Q-Networks (Function Approximation)

Instead of looking up precise values in a table, we construct a Deep Neural Network parameterized by weights θ . This network takes the environment's state vector as input and outputs estimated Q-values for all possible actions:

$$Q(s, a; \theta) \approx q_*(s, a)$$

- **Input Layer (s):** Receives the numerical features describing the state (e.g., cart position, cart velocity, pole angle, and pole angular velocity in CartPole).
- **Output Layer:** Generates a vector containing one Q-value per discrete action available. The agent can then select actions using an ϵ -greedy exploration policy based on these outputs.
- **Optimization Process:** Rather than updating individual table cells using exact Bellman backups, we adjust the network weights θ via Gradient Descent to minimize prediction errors over time.

3 The Core Challenges of Naive Deep Reinforcement Learning

Directly combining reinforcement learning with deep neural networks through standard supervised learning loss functions fails immediately due to two critical mathematical issues:

1. **Correlated Sequential Data:** In standard supervised learning, training samples must be independent and identically distributed (i.i.d.). In RL, transitions generated sequentially in an episode (state, action, reward, next state) are heavily correlated, causing the neural network to overfit to localized trajectories.
2. **Non-Stationary Targets (Moving Targets):** In standard regression, the target value is fixed. In RL, the target depends on the network itself: $R + \gamma \max_{a'} Q(s', a'; \theta)$. As we update weights θ to improve predictions, we simultaneously shift the targets we are trying to reach, leading to catastrophic divergence and infinite loops of training instability.

4 Breakthrough Stabilizing Mechanisms in DQN

Deep Q-Networks (DQN) solve these stability issues by introducing two engineering mechanisms:

4.1 1. Experience Replay (The Replay Buffer)

A memory buffer $\mathcal{D} = \{e_1, e_2, \dots, e_N\}$ stores historical transition tuples $e_t = (s_t, a_t, r_t, s_{t+1}, done)$ collected across multiple episodes.

- Instead of training immediately on the most recent step, the agent samples random mini-batches of transitions from this buffer.
- Random sampling breaks temporal correlations between consecutive steps, transforming the data distribution closer to the i.i.d. assumption required by neural networks.
- It also increases data efficiency by allowing past experiences to be reused multiple times.

4.2 2. The Target Network

To fix the moving-target problem, DQN decouples prediction from evaluation by maintaining two separate networks:

- **Main Network** ($Q(s, a; \theta)$): Actively updated at every training step to minimize the loss function.
- **Target Network** ($Q(s, a; \theta^-)$): A secondary network whose parameters θ^- are frozen and only updated by copying weights from the main network every C steps.

This keeps target values stable during intermediate optimization steps.

5 DQN Loss Function and Gradient Optimization

At each optimization iteration i , DQN minimizes the Mean Squared Error (MSE) between the target values and the predicted Q-values over sampled mini-batches:

$$\mathcal{L}_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(y_i - Q(s, a; \theta_i))^2 \right]$$

where the target value y_i calculated using the frozen target network parameters θ^- is defined as:

$$y_i = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

Taking the partial derivative (gradient) of this loss function with respect to the active network weights θ_i provides the direction for gradient descent optimization:

$$\nabla_{\theta_i} \mathcal{L}_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$